HoloAssist – “Futurino UR3eno”

Client: Augustine Le

Coach: Tan Huynh

Revised: 16/4/26

Group Name: Thu_9_12_00

Team Members:

Full Name	Student ID	Expertise
John Chen	24837332	Perception pipeline, RGB-D camera integration, object detection & classification, ROS 2 publish, coordinate frame calibration
Sebastian Baudille	24804940	XR scene & visualisation, Unity 6 environment, virtual object overlay system, UI/UX, coordinate frame alignment
Oliver Lau	24770051	Autonomous sorting, MoveIt 2 trajectory planning, pick-and-place execution, gripper integration, mode switching
Nicholas Sabatta	24796955	Teleoperation & interaction, XR controller input, RMRC/Jacobian control, gripper control, e-stop dashboard, session logging

Overview

1.1 Project Description

HoloAssist is an XR-assisted robotic sorting system. A UR3e collaborative arm, controlled via a Meta Quest 3 headset, sorts physical objects into bins within a defined workspace. An Intel RealSense depth camera detects objects as they enter the workspace, identifies them by colour or computer vision, and tracks their position until the robot manipulates them.

The system operates in two modes:

- **Teleoperation Mode — a human operator wearing the Quest 3 headset controls the robot arm and gripper to pick up objects and place them in the correct bin. The operator sees the workspace in mixed reality with XR overlays showing object identity, bin assignments, and robot state.**
- **Autonomous Mode — the robot autonomously sorts objects using MoveIt 2 planned trajectories without human input. Object-bin assignments come from a predefined mapping.**

1.2 XR Integration

Real-world objects are mapped to different virtual objects in the XR scene. For example, a physical tomato on the workspace appears as a bomb in the Quest 3 headset that the operator must "defuse" by sorting it into the correct bin. This creates an engaging, gamified operator experience while the underlying robotic task remains a practical sorting pipeline.

1.3 Stretch Goal — Human vs CPU

Teleoperation session data (object poses, grasp strategies, sort times) can be used to enable a "race mode" where a teleoperator competes against an overlay of the autonomous agent performing the same task — primarily for demonstration purposes.

2. System Architecture

2.1 High-Level Data Flow

The system is structured around four integrated subsystems communicating via ROS 2:

- Object enters workspace — the depth camera detects, identifies, and publishes its pose to ROS 2.
- Pose broadcast — the object pose is transformed into the robot coordinate frame (via QR code calibration) and forwarded to Unity and RViz.
- XR rendering — Unity receives the pose and renders the corresponding virtual object at the correct location in the operator's mixed reality view.
- Sorting — in teleop mode the operator drives the robot to the object; in autonomous mode MoveIt 2 plans and executes the pick-and-place trajectory.
- Bin verification — the depth camera confirms whether the object was placed in the correct bin.

2.2 Subsystems

Subsystem 1 — Perception and Mapping (Lead: John Chen)

Object detection, identification, and spatial tracking using the Intel RealSense depth camera.

- [ ] Configure and calibrate the RealSense (intrinsics, extrinsics, camera-to-robot transform via QR codes)
- [ ] Detect objects entering the workspace via depth segmentation and background subtraction
- [ ] Identify objects by colour, shape, or computer vision classification
- [ ] Publish object poses to ROS 2 in the robot coordinate frame
- [ ] Track objects through the workspace until manipulated
- [ ] Optionally verify correct bin placement after sorting

Subsystem 2 — Autonomous Sorting (Lead: Oliver Lau)

Autonomous pick-and-place using MoveIt 2 trajectory planning.

- [ ] Configure MoveIt 2 for the UR3e (collision geometry, planning groups, end-effector)
- [ ] Plan collision-free trajectories from current pose to detected object pose
- [ ] Execute pick-and-place sequences: approach → grip → lift → move to bin → release
- [ ] Integrate gripper open/close commands into the motion pipeline
- [ ] Handle object-to-bin assignment logic (predefined mapping)
- [ ] Support mode switching between teleoperation and autonomous sorting

Subsystem 3 — Teleoperation and Interaction (Lead: Nicholas Sabatta)

XR-based teleoperation — the human operator controls the robot arm and gripper through the Quest 3 headset.

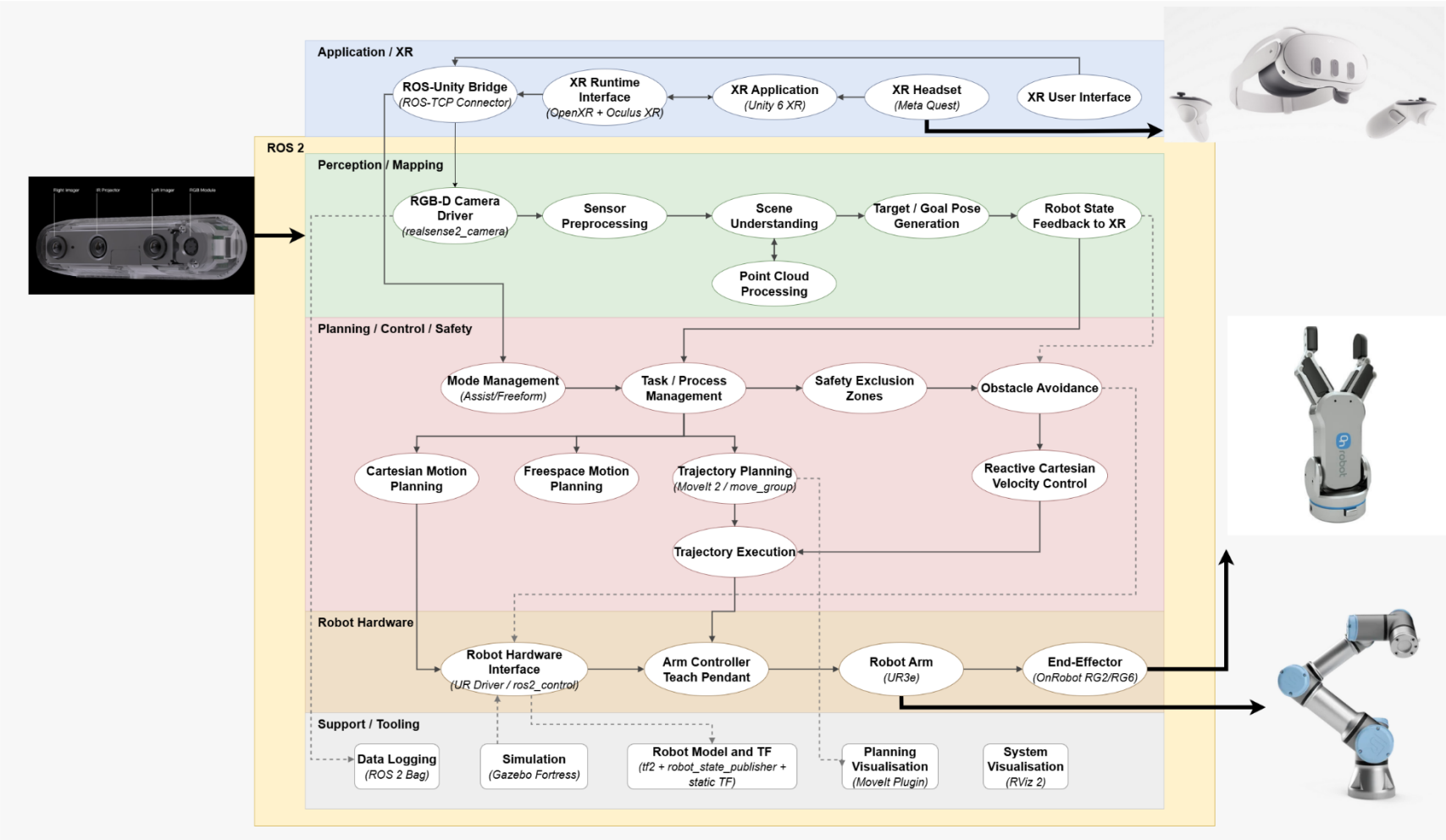
- [ ] Three teleoperation modes (existing, fully functional):
  - [ ] RMRC — Jacobian-based Cartesian velocity control with translate/rotate sub-modes
  - [ ] Direct Joint — individual joint jogging
  - [ ] Hand Guide — controller position tracking with Jacobian IK
- [ ] New: Gripper control — trigger or button to open/close the OnRobot gripper via ROS
- [ ] E-stop dashboard on Steam Deck (existing — fully functional)
- [ ] Session logging and telemetry (existing — mode tracking, durations, events)
- [ ] Safety features: joint speed limits, velocity scaling, e-stop

Subsystem 4 — XR Scene and Visualisation (Lead: Sebastian Baudille)

Unity scene rendering, XR environment, and the virtual object overlay system.

- [ ] Design and build the XR scene environment in Unity 6
- [ ] Implement the object mapping system: real object type → virtual XR object (e.g. tomato → bomb)
- [ ] Render virtual objects at the correct pose in the MR passthrough view
- [ ] Visualise bins/targets in XR (labels, highlights, placement guides)
- [ ] Coordinate frame alignment between Unity scene and real workspace via QR code anchoring
- [ ] UI/UX for the operator: bin assignments, sorting progress, task status overlays
- [ ] Visual feedback on sort correctness (success/failure indicators)

1.2 High-level System Diagram



Project Deliverables

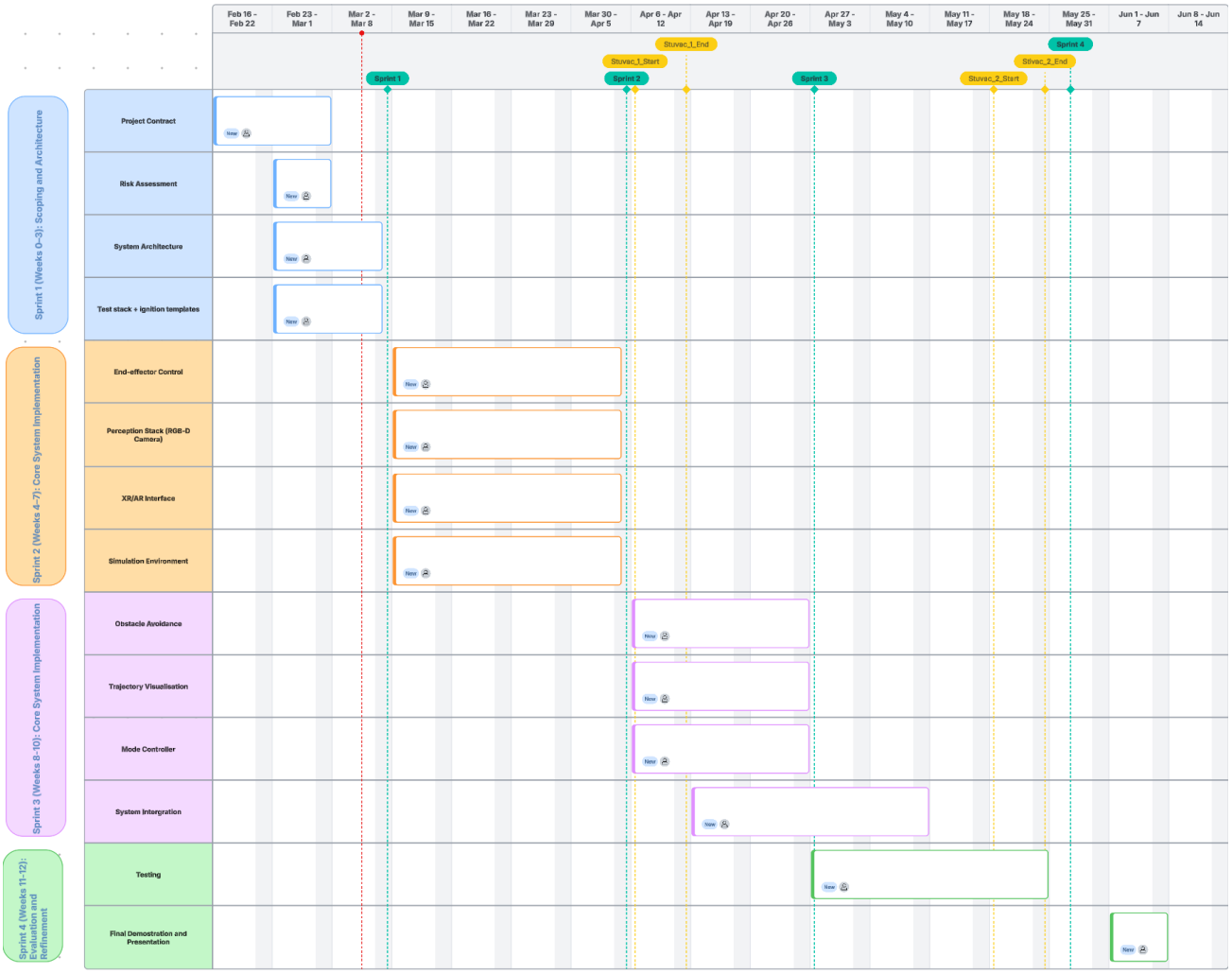
ID	Deliverable	Description
D1	End-Effector Control	Reactive Cartesian velocity control (RMRC) enabling safe teleoperation of the UR3e, including joint speed limits, velocity scaling, and safe stop on communication loss.
D2	Perception Pipeline	Object detection, identification (colour/CV), and pose estimation using the Intel RealSense RGB-D camera, publishing to ROS 2 in the robot coordinate frame.
D3	Autonomous Sorting	MoveIt 2–based pick-and-place pipeline that autonomously sorts detected objects into assigned bins without human input.
D4	XR/AR Interface	Meta Quest 3 mixed reality interface displaying robot state, virtual object overlays (real→virtual mapping), bin assignments, and task status.
D5	Mode Management	Framework supporting teleoperation mode, autonomous sorting mode, and mode switching; including e-stop dashboard on Steam Deck.
D6	Demonstration	Integrated system demonstration performing a validated sorting task in both teleop and autonomous modes.
D7	Testing	Quantitative evaluation measuring sort accuracy, task completion time, safety interventions, and teleop vs autonomous performance comparison.
D8	Documentation	Complete system architecture document, subsystem interface definitions, and risk assessment aligned with studio requirements.

1.3 Key Milestones

1.3.1 Sprint 1 (Weeks 0–3): Scoping and Architecture

Phase	Key Tasks
Phase 1 — Foundation (largely complete)	UR3e digital twin in Unity; XR teleoperation (RMRC, Direct Joint, Hand Guide); e-stop dashboard on Steam Deck; session logging and telemetry; headset streaming to dashboard. Remaining: OnRobot gripper driver in ROS; gripper control from XR controller.
Phase 2 — Perception Pipeline	RealSense configured and publishing in ROS; camera-to-robot calibration via QR codes; object detection and classification (colour/CV); object pose published in robot coordinate frame.
Phase 3 — Sorting Task	Object poses received in Unity and rendered as XR overlays; teleop sorting (operator picks and places into bins); autonomous sorting (MoveIt 2 pick-and-place); bin verification via depth camera.
Phase 4 — Integration & Polish	Mode switching between teleop and autonomous; XR object mapping (real→virtual, e.g. tomato→bomb); gamification overlay — Human vs CPU race (stretch goal); WiFi dropout auto e-stop (stretch goal); final integrated demonstration.

Time-Action Plan (Gantt Chart)



6.2 Software

ID	Software	Purpose
S1	Ubuntu 22.04 + ROS 2 Humble	Core OS and robotics middleware
S2	MoveIt 2	Autonomous trajectory planning and execution for pick-and-place
S3	UR ROS2 Driver	Interface between UR3e controller and ROS 2
S4	Unity 6	XR scene development, virtual object rendering, operator UI
S5	ROS–Unity Bridge	Bidirectional communication between ROS 2 and Unity (ros_tcp_endpoint)
S6	RealSense ROS2 Packages	RGB-D data acquisition, point cloud generation
S7	OpenCV / PCL	Image processing, object classification, point cloud filtering
S8	Gazebo Fortress	Simulation for development and testing without hardware
S9	rosbag2	Data logging for session replay, testing, and training data capture
S10	Git / GitHub	Version control and collaborative development

1.4 Learning Goals

1.4.1 Team Learning Goals

As a team, we aim to:

- Apply systems engineering principles to design, integrate, and validate a multi-subsystem robotics solution.
- Develop and evaluate a robotics path planning and control framework within a real-world collaborative context.
- Strengthen interdisciplinary collaboration through clearly defined subsystem ownership and interface agreements.
- Improve technical communication through sprint reviews, documentation, and reflective critique.
- Develop resilience in debugging and integration of complex robotic systems.

1.4.2 Individual Learning Goals

Each team member aims to deepen expertise within their subsystem while strengthening integration awareness:

- Develop advanced understanding of reactive motion control and constraint-based robotics.
- Gain practical experience integrating perception data into real-time control frameworks.
- Improve XR-based interface design for robotics applications.
- Strengthen hardware–software integration and safety-critical system thinking.
- Enhance professional capabilities in teamwork, reflection, and iterative design improvement.

Resources

The successful completion of this project is incumbent on access to robotics hardware, perception systems, XR equipment, fabrication facilities, and supporting software frameworks. The required resources are outlined below.

1.5 Hardware and Software (Laboratory Provided)

ID	Hardware	Quantity	Description
H1	UR3e Collaborative Robot	1	6-DOF Universal Robots UR3e collaborative manipulator used as the primary robotic platform for teleoperation, velocity control implementation, and collaborative task execution.
H2	Mobile Robot Workstation	1	Trolley-mounted UR3e setup, including base frame and workspace surface, enabling flexible repositioning within the lab environment while maintaining safety compliance.
H3	UR3e Teach Pendant	1	Official Universal Robots teach pendant used for robot configuration, safety setup, emergency stops, manual jogging, and recovery operations during development and testing.
H4	Intel RealSense RGB-D Camera	1	RGB-D depth camera used for real-time point cloud generation, workspace sensing, and dynamic safety boundary estimation.
H5	Camera Mount / Tripod	1	Adjustable tripod or mounting bracket for stable positioning of the RGB-D camera in either eye-in-hand or eye-to-hand configuration.
H6	Parallel Gripper	1	Two-finger end-effector for executing simple collaborative manipulation tasks (e.g., object alignment, placement, or handover demonstrations).
H7	Workstation PC	1	High-performance desktop workstation running ROS 2, perception processing, control stack, and XR communication bridge.
H8	Meta Quest 2 Headset	1	XR headset used for augmented reality visualisation of robot pose, point clouds, trajectories, and safety boundaries.
H9	USB-C to Ethernet Adapter	1	An adapter that enables stable wired network connectivity for the Meta Quest headset, reducing wireless latency and improving communication reliability.
H10	Ethernet Cables	2–3	CAT6 Ethernet cables for direct communication between workstation, UR3e controller, networking hardware, and XR device.

ID	Software	Description
S1	Ubuntu 22.04 LTS & ROS 2 Humble	Core operating system and robotics middleware used for communication between perception, control, and XR subsystems.
S2	MoveIt 2	Motion planning framework used for robot modelling, kinematics, and optional trajectory validation.
S3	Universal Robots ROS2 Driver	Interface enabling communication between the UR3e robot controller and the ROS 2 environment.
S4	RViz2	3D visualisation tool used for debugging robot states, transforms, trajectories, and point cloud data.
S5	RealSense ROS2 Packages	Drivers for acquiring RGB-D data and generating point clouds from the Intel RealSense camera.
S6	Gazebo	A simulation tool to support the visualisation and testing of the system virtually without hardware.
S7	Unity 6	A game engine is used to develop the XR interface and visualise robot state and workspace information.
S8	Unity XR Interaction Toolkit	Toolkit used within Unity to handle headset tracking and controller input for XR interaction.
S9	ROS–Unity Communication Bridge	Middleware enabling communication between ROS 2 nodes and the Unity XR environment.
S10	tf2	ROS transform library used to manage coordinate frames between the robot, camera, and XR reference systems.
S11	PCL (Point Cloud Library)	Used for filtering and processing real-time point cloud data.
S12	OpenCV	Image processing library for filtering and validating RGB-D image data.
S13	rosbag2	A logging tool that supports the record of system data for testing, debugging, and evaluation.
S14	Git and GitHub	Version control platform used for collaborative development, code management, and subsystem integration.

1.6 Modifications to Laboratory Hardware

ID	Modification	Description	Purpose
M1	Custom Tool Holder / Gripper Adapter	Design and fabrication of a task-specific attachment compatible with the UR3e parallel gripper.	Enables execution of the selected demonstration task and mimics a simplified industry-relevant application.
M2	RGB-D Camera Mounting Bracket	Design and fabrication of a secure mounting solution for the Intel RealSense camera. Mount location may be on the robot base, workspace frame, or end-effector, depending on chosen configuration (eye-in-hand or eye-to-hand).	Ensures stable depth sensing and accurate spatial alignment for perception and safety boundary generation.

All modifications will comply with laboratory safety procedures and be reviewed with the lab manager before being added to the existing lab hardware.

1.7 Hardware and Software (Team-Sourced)

ID	Item	Type	Description
TS1	Meta Quest 2	Hardware	XR headset used for augmented reality visualisation and teleoperation interface testing.
TS2	Meta Quest 3	Hardware	An optional upgraded XR headset is used to improve tracking stability and display performance during testing and demonstrations.
TS3	Unity Hub & Unity 6	Software	Development environment for building and managing the XR interface and spatial rendering pipeline.
TS4	ROS–Unity Communication Bridge	Software	Middleware enabling bidirectional communication between ROS 2 nodes and the Unity XR environment (e.g. ROS-TCP-Connector or equivalent).
TS5	Dedicated Networking Hardware (Optional)	Hardware	An optional router or wired networking setup to improve communication stability and reduce latency between the XR device and the control workstation.

7. System Evaluation

Level	Criteria
Pass (MVP)	XR headset displays stable robot model and end-effector pose. Operator can teleoperate robot using reactive velocity control. System stops safely on communication loss. Depth camera publishes to desktop UI. Operator can observe pose markers/transforms in XR.
Credit	Autonomous mode operational — MoveIt 2 plans and executes basic pick-and-place. Teleop sorting mode achievable in repeated trials via XR input. Gripper open/close integrated from XR controller.
Distinction	XR virtual object mapping operational (e.g. tomato → bomb). Mode switching between teleop and autonomous validated. Demonstration task completed consistently under defined time threshold. Passthrough / spatial marker visualisation working.
High Distinction	Human vs CPU race mode overlay functional. Quantitative comparison between teleop and autonomous modes (sort time, accuracy). Mixed reality UI visualises and debugs depth camera feed. WiFi dropout auto e-stop implemented.

8. Subsystem Evaluation

Subsystem 1 — Perception (Lead: John Chen)

Object detection, identification, and spatial tracking using the Intel RealSense depth camera. Provides the perception pipeline that feeds both the teleoperation and autonomous sorting modes.

Grade	Summary	Criteria
P	Camera publishing in ROS	☐ RealSense ROS 2 wrapper installed and launching reliably ☐ Colour and depth image topics publishing at expected rates ☐ Point cloud topic available ☐ Camera feed verified in RViz
C	Object detection and segmentation	☐ Background subtraction or depth thresholding isolates objects on the workspace surface ☐ Detected object regions are identifiable (bounding box or centroid) ☐ Detection works for predefined objects placed on the workspace
D	Classification and pose in robot frame	☐ Object classification distinguishes between the predefined object set (e.g., by colour or shape) ☐ QR code / AprilTag calibration transforms camera-frame detections into robot-frame poses ☐ Object type and pose published as ROS messages usable by Unity and the autonomous sorting node ☐ Calibration is repeatable and does not require manual tuning each session
HD	Real-time tracking and bin verification	☐ Continuous tracking of detected objects — objects tracked from entry until manipulation ☐ Object identity persists across frames (same object is not re-detected as new) ☐ After a sort, camera verifies the object is in the expected bin and publishes success/failure confirmation ☐ System handles multiple objects in the workspace simultaneously

Subsystem 2 — Autonomous Sorting (Lead: Oliver Lau)

Autonomous pick-and-place sorting using MoveIt 2 trajectory planning. The robot sorts detected objects into the correct bins without human input.

Grade	Summary	Criteria
P	MoveIt 2 in simulation	☐ MoveIt 2 planning group configured for UR3e (arm) ☐ Trajectory planning and execution demonstrated in RViz simulation
C	Collision detection + gripper driver	☐ Gripper opens/closes as part of a scripted sequence (approach → grip → lift) ☐ Collision boundaries (at minimum, the workspace table) prevent unsafe trajectories ☐ OnRobot gripper ROS driver functional — open/close commands work
D	Full pick-and-place from perception	☐ Subscribes to object poses from the perception pipeline ☐ Full pick-and-place cycle: approach → grip → lift → move to bin → release ☐ Object-to-bin mapping is defined (e.g., config file or lookup table) ☐ Sequence is repeatable across multiple objects and trials ☐ Mode switching between teleop and autonomous is functional
HD	Multi-object sessions with metrics	☐ Multiple objects sorted in a single autonomous session without manual resets ☐ Bin locations are parameterised — planner targets the correct bin based on object type ☐ Collision avoidance includes bins, workspace boundaries, and known obstacles ☐ Sorting metrics logged: objects sorted, success/failure per object, cycle time per sort ☐ Robust to minor pose estimation errors (approach strategy handles small offsets)

Subsystem 3 — Teleoperation and Interaction (Lead: Nicholas Sabatta)

XR-based teleoperation of the UR3e arm and OnRobot gripper via Meta Quest 3. Includes control modes, operator dashboard, session logging, and gripper integration.

Grade	Summary	Criteria / Evidence
P	Stable digital twin on Quest 3	☐ JointStateSubscriber.cs subscribes to /joint_states and drives Unity joint transforms ☐ Digital twin verified on Quest 3 — joints match real robot direction and axes ☐ Purely kinematic (no physics simulation) — stable under all conditions
C	RMRC + Direct Joint + e-stop dashboard	☐ RobotController.cs with RMRC (Jacobian-based Cartesian velocity, translate/rotate sub-modes) and Direct Joint mode ☐ Both modes publish to /forward_velocity_controller/commands at 50 Hz ☐ UR3eKinematics.cs provides forward kinematics, geometric Jacobian, and DLS pseudoinverse ☐ Dashboard (main.py + ros_interface.py) with e-stop: burst zero-velocity publish, controller deactivation, 5-second hold resume, cooldown
D	Three modes + headset streaming dashboard	☐ Three control modes cycled via Menu button: RMRC → Direct Joint → Hand Guide ☐ RMRC has two sub-modes (Translate/Rotate) toggled via X button ☐ HeadsetStreamPublisher.cs renders VR camera view → JPEG → /headset/image_compressed at 15 FPS ☐ Dashboard tabs: STATUS, HEADSET (live XR view), STATS (session bar + joint velocity graph), LATENCY, SESSION ☐ Launch scripts (launch.sh, dashboard.sh) for one-command startup
HD	Hand Guide mode + session logging + gripper	☐ Hand Guide mode: hold right grip to engage — controller 3D position mapped to end-effector via proportional control + Jacobian IK, gain 2, speed cap 0.15 m/s ☐ SessionLogger.cs: tracks mode switches, per-mode durations, session time — publishes JSON to /session/status (2 Hz) and saves on quit ☐ Dashboard saves session log to ~/holoassist_sessions/ on shutdown (events, e-stop count, Unity session info) ☐ OnRobot gripper open/close triggered from XR controller during teleoperation, enabling full pick-and-place workflow

Subsystem 4 — XR Scene and Visualisation (Lead: Sebastian Baudille)

Unity XR environment design, virtual object overlay system, and operator-facing visualisation. Responsible for rendering the mixed reality scene on the Quest 3, including the real-to-virtual object mapping that creates the gamified sorting experience.

Current:

Subsystem 4 – **XR and Scene Visualisation**	**Lead by: Sebastian**
P	Operator can observe pose markers and robot transforms in XR or desktop visualisation.
C	System visualisation supports repeated demonstration trials by clearly presenting robot state, task progress, and safety context to the operator.
D	Demonstration task is completed consistently within a defined time threshold, supported by integrated XR scene feedback and visual cues.
HD	Visual outputs support quantitative comparison between freeform and assisted modes, including clear representation of active safety regions, control state, or performance overlays.
Perfect	Final XR scene integration is polished and showcase-ready, including stable trajectory preview visualisation in mixed reality and robust behaviour under simulated failure conditions.

Proposed:

Grade	Summary	Criteria
P	XR Unity UI	• XR user can observe pose markers and basic robot transforms information within the unity XR UI.
C	XR UI and Environement	• System visualisation supports repeated demonstration trials by clearly presenting robot state, task progress, and safety context to the operator. • Subscribes to object pose data from ROS (via ros_tcp_endpoint)
D	Real-to-virtual mapping + bin visualisation	• Mapping system translates object type from perception (e.g., "tomato") to a virtual representation (e.g., bomb 3D model) • Virtual objects are visually distinct and clearly identifiable by the operator • Bin locations rendered in XR with labels, colours, or icons indicating target object type

		• Operator can see at a glance which object goes where
HD	Polished UX with progress tracking and feedback	• Sorting progress UI: objects remaining, objects sorted, current score or tally • Visual feedback on sort outcome — success (green flash, checkmark) and failure (red flash, warning) on bin placement • Task guidance overlays: highlights on the next object to sort, directional indicators toward target bin • Scene is visually polished — consistent art style, readable UI at arm's length, no placeholder assets in final build • Smooth performance on Quest 3 with all overlays active

9. Learning Goals

9.1 Team Learning Goals

- ☐ Apply systems engineering principles to design, integrate, and validate a multi-subsystem robotics solution.
- ☐ Develop and evaluate a full robotic sorting pipeline combining perception, planning, teleoperation, and XR.
- ☐ Strengthen interdisciplinary collaboration through clearly defined subsystem ownership and interface agreements.
- ☐ Improve technical communication through sprint reviews, documentation, and reflective critique.
- ☐ Develop resilience in debugging and integration of complex robotic systems.

9.2 Individual Learning Goals

- ☐ Gain practical experience integrating RGB-D perception data into real-time control and planning frameworks.
- ☐ Develop advanced understanding of autonomous pick-and-place using MoveIt 2 and collision-aware planning.
- ☐ Improve XR-based interface design for robotics applications, including gamification and operator UX.
- ☐ Strengthen hardware–software integration and safety-critical system thinking.

- Enhance professional capabilities in teamwork, reflection, and iterative design improvement.

Appendix

Appendix A

Other potential names: HoloMotion, VisionLayer, Spatial Robotics, OmniSpace, Telessist, TeleMotion, HoloLens